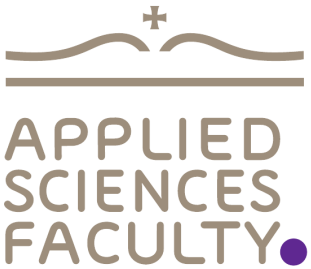
Співпрограми (coroutines) в C++





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Що таке співпрограми

- Coroutine (співпрограма, за аналогією з ”підпрограма”) – узагальнення поняття підпрограми (функції).
- Це підпрограма, яка може призупинятися та продовжуватися декілька раз,
  - зберігаючи про цьому весь локальний контекст – контекст процесора та локальні змінні.
- Завдяки цьому вони є зручною абстракцією для кооперативної багатозадачності.
  - Тому, хоча ідея дуже давня (запропоновано не пізніше 1963-го), зараз знову набула великої популярності.

Класифікація співпрограм

- Асиметричні співпрограми не можуть керувати, до якого кадру стеку перемикатися – переходить лише до того, з якого було викликано співпрограму. Зазвичай, надаються ключові слова типу yield. Більш звичні – більш схожі на звичайні функції.
  - Симетричні можуть вказувати, до якого кадру стеку переходити.
- **Stackful coroutine** – допускають вкладеність. Зазвичай, використовують блок пам'яті на купі в ролі стеку.
  - Зазвичай – симетричні.
- **Stackless coroutines** – не допускають, мають бути поміченими спеціальними засобами мови[1].
  - Під час виконання використовують стек того, хто викликав, але під час переключення, зберігають стан на купі. Тому, можуть різні стадії свого виконання здійснювати на різних стеках, може навіть у різних потоках.
  - Зазвичай – асиметричні.

[1] Наприклад, ключовими словами.

Порівняння з іншими засобами

- Функції є частковим випадком співпрограм.
- Співпрограми, за функціональністю, схожі на потоки – вони надають concurrency, але не надають parallelism.
  - Можуть бути більш передбачувані та економні з точки зору часу перемикання та не потребувати підтримки операційної системи. Також, можуть не потребувати явної синхронізації (хоча проектування коду для цього ускладнюється). Це особливо важливо в системах реального часу.
- Генератори – частковий випадок співпрограм.
- Таким чином, співпрограми зручні для:
  - Створення скінчених автоматів – завдяки стану, який зберігають.
  - Кооперативної багатозадачності.
  - Зокрема: в мережевих серверах – зокрема, для побудови ефективної реалізації відповідних скінчених автоматів реакції на події, чи комп'ютерних іграх, для опрацювання реакції об'єктів.

Реалізація в C++

- Boost містить stackfull-співпрограми, Coroutine2.
  - Лише асиметричні[2].
- C++20 додав підтримку stackless-співпрограм.
  - Однак, лише мінімальну підтримку у мову – підтримки в стандартній бібліотеці поки немає (станом на C++26).
  - Є асиметричними.
  - Компілятор може оптимізувати використання пам’яті для, наприклад, повністю вкладених співпрограм.

[2] Застаріла Boost.coroutine надає і симетричні.

Співпрограми C++20

- Додано три нових ключових слова:
  - **co_await** – унарний оператор, передає керування тому, хто викликав[3], поки не завершаться обчислення в аргументі. Тут співпрограма очікуватиме на вхідні дані або якусь подію.
  - **co_yield** – унарний оператор, призупиняє виконання співпрограми і повертає у місце виклику операнду. Цією конструкцією співпрограма повертає значення.
  - **co_return** – унарний оператор, завершення виконання співпрограми, після якого вона не може бути продовженою. Може повертати тип, відмінний від того, що повертав **co_yield**. Якщо в співпрограмі його немає – додається неявно.
- Співпрограми C++20 є реалізованими самою мовою, але щоб функція вважалася співпрограмою, достатньо, щоб в її коді трапився один з цих операторів.
  - Функція, що використовує return не може бути співпрограмою. Не можуть нею бути конструктори та деструктори, constexpr-функції, функції, типу результатів яких позначено як auto, функції зі змінною кількістю аргументів, main().

[3] Часто це може бути той чи інший диспетчер.

Щодо гнучкості співпрограм

- Однак, для використання співпрограм, потрібно надати багато інформації – створивши відповідні типи[4].
  - В C++20, в стандартній бібліотеці немає жодного, тому потрібно або писати величезні простирадла коду лише щоб почати їх використовувати, або скористатися сторонніми бібліотеками.
  - C++23 додав std::generator – дуже спеціалізований приклад співпрограми.
- Ці типи визначають:
  - Чи співпрограма повинна зразу почати виконуватися, чи бути призупиненою на початку.
  - Які типи можуть отримувати оператори co_await, co_yield, co_return, як поводитися з відповідними значеннями.
  - Які звертання до co_await та co_yield призупиняють виконання, а які – ні.
  - Коли співпрограма призупинена, чи повертатися до того, хто її викликав, чи викликати іншу співпрограму.

[4] Як там говорять про Лінукс: ”Ти можеш все сконфігурувати! І ти, курча, будеш все конфігурувати!”.

Типи, пов'язані з співпрограмами

- Дескриптор співпрограми – надається компілятором. Решта потрібно надати самостійно.
- Тип **promise**[5] – призначений для збереження стану співпрограми, і зберігати дані, які співпрограма yield.
  - Він вказує поведінку за першого входу, під час виходу, у відповідь на co_yield чи co_await та деякі інші аспекти "життя" співпрограми.
- Awaiter – типи, які конструюються promise для всіх викликів co_await, co_yield, co_return і визначають їх поведінку.
  - Певна їх кількість є в C++20, наприклад: std::suspend_always та std::suspend_never.
- Типу-дескриптор співпрограми – контейнер для promise, власне, тип, видимий користувачеві.
  - В якомусь сенсі, цей тип можна називати future для співпрограм.
- Засоби, надані мовою, знаходяться в <coroutine>.

[5] Тут є ідейна аналогія з std::promise, але крім такої схожості, ці типи ніяк не пов'язані.

Приклад – 1

- Приклад з cppcore[6] – експериментальної бібліотеки підтримки співпрограм.

```
cppcoro::generator<uint64_t> fibonacci() {
    uint64_t a = 0, b = 1;
    while (true) {
        co_yield b;
        auto tmp = a;    a = b;   b += tmp;
    }
}
.........................
    for (auto i : fibonacci()) {
        if (i > 1000) break;
        std::cout << i << ",␣";
    }
```

- Для об’єкту, що повертається співпрограмою, begin() розпочинає виконання співпрограми, до co_yield або завершення.
- Повертає посилання на результат до co_yield або завершення.
- Оператор ++ продовжує виконання.
- ◇ Іншими словами, promise цього об’єкту веде себе як ітератор.
- C++23 підтримує схожу конструкцію – std::generator, станом на березень 2025, підтримується GCC та MSVC.
- ◇ З блогу MSVC: ”std::generator: Standard Library Coroutine Support”.

[6] Використано цей форк cppcoro: `https://github.com/andreasbuhr/cppcoro` – сама бібліотека трохи закинута.

Приклад – 2

- Задача producer-consumer теж гарно виражається через співпрограми[7]:

```
cppcoro::task<> consume1(uint64_t n) {
    uint64_t i = 1;
    for(auto const& s : fibonacci()) {
        std::cout << "" << s << ",␣";
        if (++i > n) break;
    }

    co_return;
}
.........................

    cppcoro::sync_wait(consume1(5));
```

- Оскільки main не може використовувати co_wait, використовуємо співпрограму cppcoro::sync_wait().
- Ремарка: cppcoro крива у всі сторони, документація і код не синхронізовані і т.д., ми використовуватимемо її лише мінімально.

[7] Володіючи concurrency, але не parallelism-ом.

Приклад – 3

- cppcoro::generator не дозволяє використовувати в тілі co_await.
- Якщо це потрібно – можна скористатися асинхронним генератором:

```
cppcoro::task<int> next_value_2() {
    co_return rand();
}
cppcoro::async_generator<int> produce2() {
    while (true) {
        auto v = co_await next_value_2();
        v *= 2;
        co_yield v;
    }
}
cppcoro::task<> consume2(int n) {
    int i = 1;
    // В стандарт не потрапило: `for co_await( const auto& s : produce_items() )'
    auto lookup = produce2();
    for(auto itr = co_await lookup.begin(); itr != lookup.end(); co_await i++tr) {
        std::cout << i*tr << ",␣";
        if (++i > n) break;
    }
}
.........................
    cppcoro::sync_wait(consume2(5));
```

Реалізація власного генератора

- Генератор – хоч і частковий випадок співпрограми, є хорошою відправною точкою для того, щоб зрозуміти роль різних частин coroutine future.

```
template<typename T>
class my_generator {
public:
  struct promise_type { // < Потрібна саме ця назва v
    using handle = std::coroutine_handle<promise_type>;

    T value;

    auto get_return_object() noexcept {          // Створюємо об'єкт
      return my_generator{handle::from_promise(*this)};
    }

   // Вказує виконувати співпрограму до co_yield або co_return зразу після створе
    std::suspend_never initial_suspend() noexcept { return {}; }

    // suspend_always -- не знищувати зразу після завершення.
    std::suspend_always final_suspend() noexcept { return {}; }

    //  co_return не повертає нічого
      void return_void() noexcept {}

    // Ігноруємо виключення
      void unhandled_exception() { std::terminate(); }
```

```
        // Поведінка 'co_yield value'
        std::suspend_always yield_value(T&& v) noexcept {
            value = std::move(v);
            return {};
        }
        std::suspend_always yield_value(const T& v) noexcept {
            value = v;
            return {};
        }
    };

        class iterator{ ....... }; // Див. наступний слайд

    // Перший результат було прочитано завдяки inital_suspend,
    // який повертає suspend_never
    iterator begin() { return iterator{coro_m.done() ? nullptr : this}; }
    iterator end() { return iterator{nullptr}; }
private:
    promise_type::handle coro_m;
    explicit my_generator(promise_type::handle coro) noexcept:  coro_m{coro} {}
};
```

```
// Підтримка для використання для ітерування
class iterator {
public:
    bool operator != (iterator second) const {
        return m_self != second.m_self;
    }

    iterator & operator++() {
        // Продовжуємо виконання співпрограми
        m_self->coro_m.resume();
        // Перевіряємо, чи співпрограма завершилася
        if (m_self->coro_m.done()) {
            m_self = nullptr;
        }
        return *this;
    }

    T operator*() {
        // Повертаємо результат, збережений в promise
        return m_self->coro_m.promise().value;
    }
private:
    iterator(my_generator *self): m_self{self} {}
    my_generator *m_self;
    friend class my_generator;
};
```

















